

## A) Algorithmique

### PARTIE 1:

1) Considérons l'algorithme suivant:

```
1 def sstmn_naif(T):  
2   max_val = 0  
3   for i in range(len(T)):  
4     for j in range(i, len(T)+1):  
5       somme = sum(T[i:j])  
6       if somme > max_val:  
7         max_val = somme  
8   return max_val
```

\* IP y a deux opérations à considérer: les additions et les comparaisons. (l'affectation de  $T[i:j]$  est du même ordre de grandeur que les additions à cet "étape").

$$\text{Donc } T(n) = \sum_{i=0}^{n-1} \left( \sum_{j=i}^n \left( \sum_{k=i}^j 1 \right) \right) \\ = \sum_{i=0}^{n-1} \sum_{j=i}^n (j-i) \in \mathcal{O}(n^3)$$

\* Correction: Pour chaque indice de départ possible, on fait la somme de chaque sous-tableau possible et si cette somme est plus grande que max\_val, on la sauvegarde dans max\_val.

Donc: - on traite bien tous les tableaux possibles  
- max\_val représente bien la somme du sous-tableau dont la somme de éléments est maximale.  
- le code fonctionne bien car on est sûr de trouver les indices st. croissants.



2)

\* l'ennemi aurait un constant:

↳ on remplace  $sum$  par une fonction  $sum(T, deb, fin)$  qui assure d'entre le recopie de tableau.

\* Complexité en temps:

Idee: on ajoute que  $TC(j)$  à chaque fois, sans refaire la somme.  
def  $sum\_naif(T)$ :

$max\_val = 0$

$current\_sum = 0$

for  $i$  in range( $len(T)$ ):

$current\_sum = 0$

for  $j$  in range( $i, len(T)$ ):

$current\_sum += TC[j]$

if  $current\_sum >= max\_val$ :

$max\_val = current\_sum$

return  $current\_sum$

PARTIE 2:

3) On suppose que  $T = T_1 + T_2$ .  
Non, voici un contre-exemple:

$T_1$ :  $\boxed{10 \ 17}$   $sum(T_1) = 27$

$T_2$ :  $\boxed{31 \ 7 \ 5}$   $sum(T_2) = 5$

Mais:  $T$ :  $\boxed{10 \ 17 \ 31 \ 7 \ 5}$   $sum(T) = 30$   
qu'on ne pourrait pas déduire de  $sum(T_1)$  et  $sum(T_2)$ .

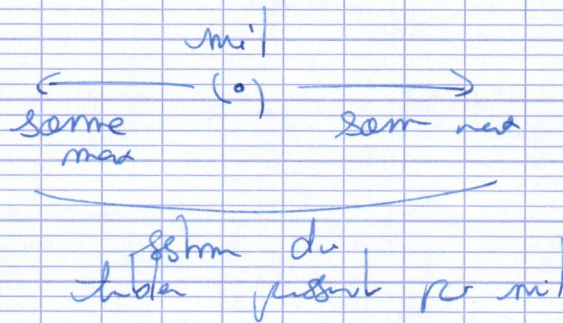


4) On veut écrire un algo de complexité linéaire  $ssrm\_aux(T, deb, mil, fin)$  qui calcule la somme maximale d'un sous-tableau  $T[i:j]$  de  $T$  avec  $deb \leq i \leq mil \leq j \leq fin$

↳ le tableau doit traverser la case d'indice "mil"

→ Idée: - regarde la somme max qui pte de mil.  
- gauche  
- droite

↳ la somme maximale du sous-tableau qui traverse le milieu est  $ssm(gauche) + ssm(droite)$



5) On a trois cas à traiter: (on peut être le max)

- traverser le milieu
- est à gauche
- est à droite

def  $ssrm\_dpr(i, deb, fin)$ :  
 if ( $fin == deb$ ): return 0  
 elif ( $fin - deb == 1$ ): return  $\max(0, T[deb])$   
 mil =  $(fin + deb) // 2$   
 # cas est. gauche & droite  
 max\_g =  $ssrm\_dpr(T, deb, mil)$   
 max\_d =  $ssrm\_dpr(T, mil, fin)$   
 # cas milieu  
 max\_mil =  $ssrm\_aux(T, deb, mil, fin)$   
 return  $\max(max\_g, max\_d, max\_mil)$

sous-tableau vide



6) On remarque assez directement que  
 $T(n) = 2T\left(\frac{n}{2}\right) + n$   
 $\in \Theta(n \log n)$ .

En effet, on dérive la récursion:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$= 2\left(2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right) + n = 2^2 T\left(\frac{n}{2^2}\right) + 2n$$

$\vdots$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

Le  $k$  minimal est  $\log_2 n$ , et on a:

$$T(\log_2 n) = \log_2 n \cdot n \in \Theta(\log n \cdot n)$$

PARTIE 3 : non traitée